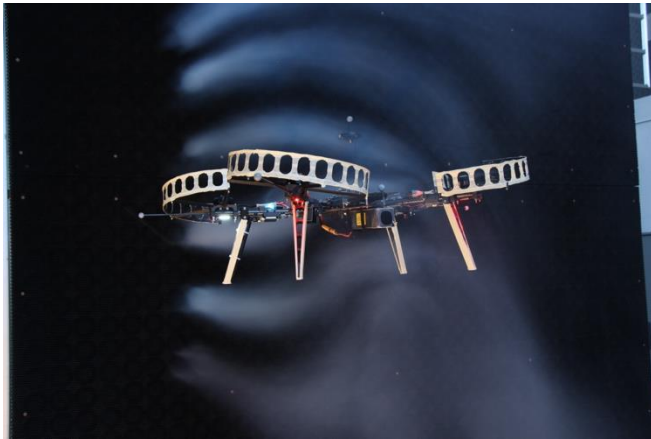
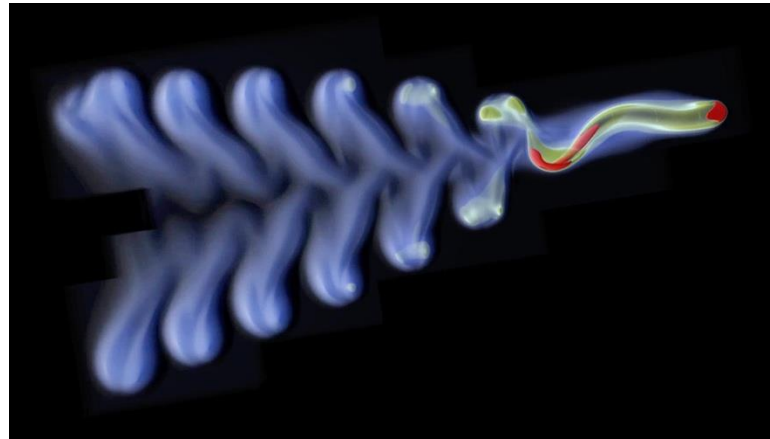


Model Learning and Deep MBRL

$$\dot{x} = \frac{dx}{dt} = f(x, u) \quad x_{t+1} = f(x_t, u_t)$$



Aerodynamics in wind, *Neural-Fly*



Fluid dynamics, MIT van Rees Lab



Offroad vehicle dynamics, UW Racer team

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Guest Lectures

❑ Please attend in person. “Almost” mandatory (strongly encourage)!



Lecture 18, 10/31 (Thu)
Aviral Kumar, CMU CSD/MLD

Offline RL: Algorithms and Applications



Lecture 22, 11/19 (Tue)
Yonatan Bisk, CMU LTI

Foundation Models in Robotics

Course Structure

Latest schedule: <https://16-831.github.io/fall24/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☐ Topic group 4: Model-free reinforcement learning
- ☒ Topic group 5: Model-based reinforcement learning
- ☐ Topic group 6: Bandit, preference-based learning, exploration
- ☐ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation

Outline of the MBRL Topic Group

- ❑ Part 1: control theory basics
 - Stability and Lyapunov methods for stability
 - Uncertainty and robust control
 - Adaptive control
 - Linear systems and continuous-time LQR
 - Linearization of nonlinear systems
- ❑ Part 2: how can we make decisions if we *know* the dynamics?
 - Trajectory optimization (TO) and model predictive control (MPC)
 - Discrete time LQR
 - Sequential quadratic/convex programming (SQP/SCP)
 - iLQR/DDP
 - Sampling-based MPC
- ❑ Part 3: learn *unknown* dynamics and use it for control
 - Model learning
 - Policy optimization with the learned model
 - Deep MBRL: Dreamer, TD-MPC, MBRL+MFRL, etc

Model-based RL and Optimal Control

- ❑ MBRL and optimal control are essentially both solving:
 - c_t is cost function
 - X_t is the state constraint (e.g., collision avoidance)
 - U_t is the control constraint (i.e., actuation limit)
 - T is the horizon
- ❑ This lecture: f_t is *unknown*
- ❑ Today we will alternatively use (x, u, c) and (s, a, r) notation systems (depends on which paper/algorithm we discuss)

$$\begin{aligned} & \min_{x, u} \sum_{t=1}^T c_t(x_t, u_t) \\ \text{s. t. } & \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{cases} \end{aligned}$$

Model Learning

- ❑ Suppose $f_t = f$ (time-invariant system)
- ❑ Can we just use learning approaches to learn a model $f_\theta(x, u)$ and use it for control?
- ❑ Yes! But need to be very careful

$$\min_{\theta} \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \|x_{t+1} - f_\theta(x_t, u_t)\|^2$$

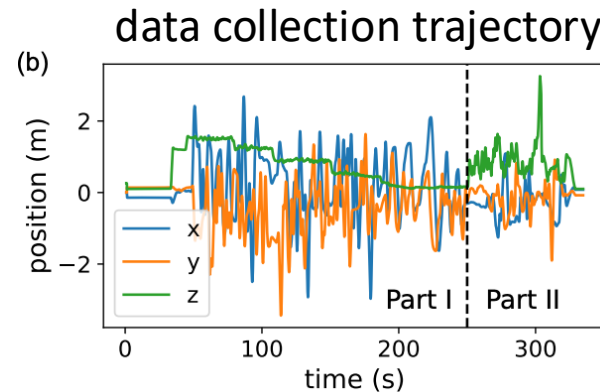
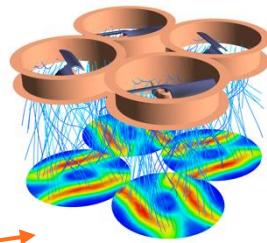
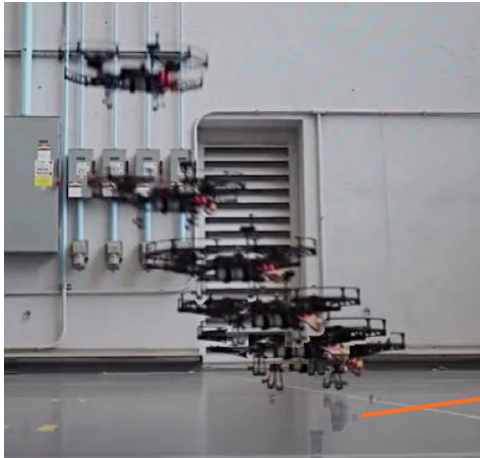
- ❑ It is a *supervised regression* problem, but two fundamental differences:
 - Data generation is governed by the underlying (unknown) dynamics
 - I.e., we lose all good properties from i.i.d. data scheme
 - What policy we should use to generate data?
 - A good model doesn't necessarily mean good or easy control/planning
 - I.e., the regression error (learning objective) is NOT what we ultimately care about
 - What function class we should use to learn?

Model Learning: Challenges

- ❑ **Challenge 1:** Domain shift, over-fitting, and generalization

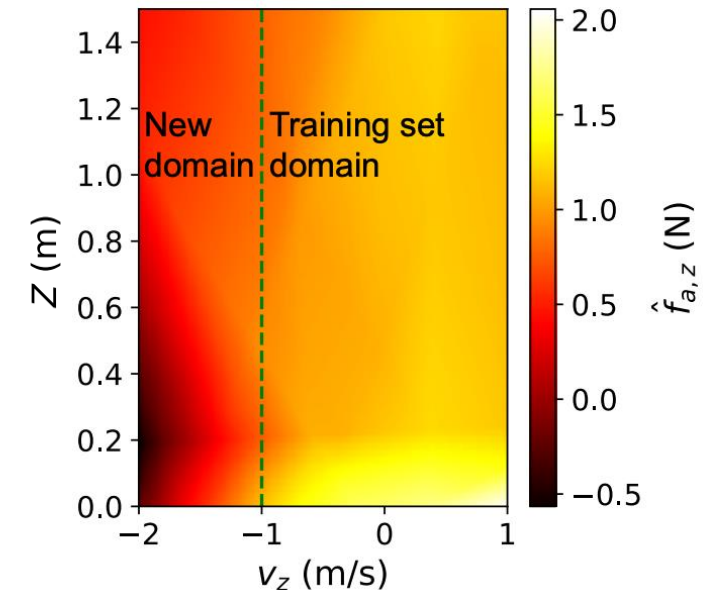
$$\min_{\theta} \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \|x_{t+1} - f_{\theta}(x_t, u_t)\|^2$$

- ❑ Example: residual dynamics learning for drone landing



**Neural Lander: Stable Drone Landing Control
Using Learned Dynamics**

learned f_{θ} using a 4-layer MLP



Model Learning: Challenges

❑ Challenge 2: “Good” model might be “bad” for control

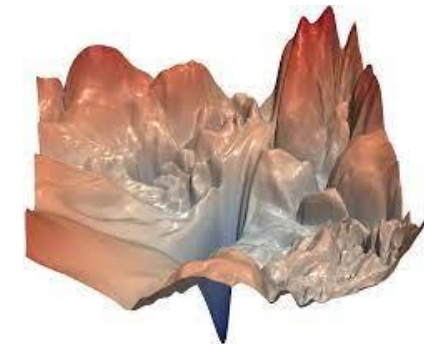
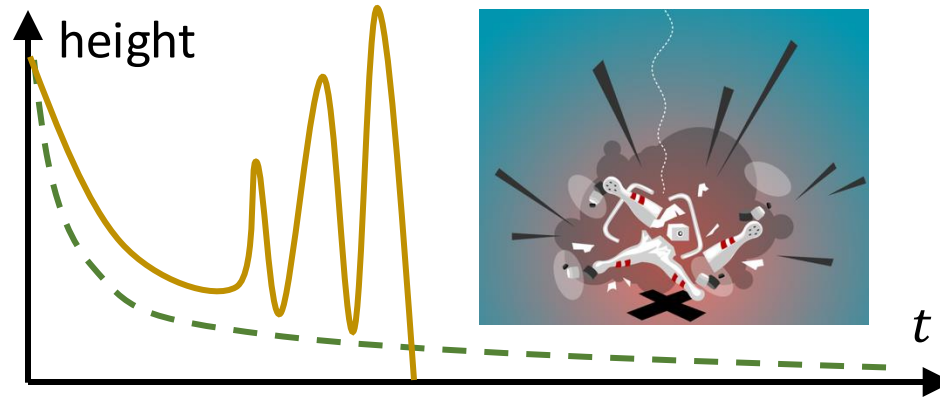
- The optimal control/planning problem might be ill-posed or hard to solve

$$\min_{\theta} \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \|x_{t+1} - f_{\theta}(x_t, u_t)\|^2$$

$$\begin{aligned} & \min_{x, u} \sum_{t=1}^T c_t(x_t, u_t) \\ \text{s. t. } & \begin{cases} x_{t+1} = f_{\theta}(x_t, u_t) \\ x_t \in X_t \quad u_t \in U_t \end{cases} \end{aligned}$$

❑ Example: residual dynamics learning for drone landing

- Controller using f_{θ} is not stable! Local “steepness” destroys stability

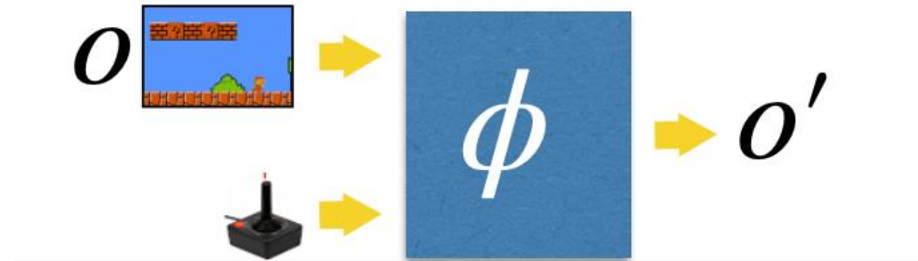
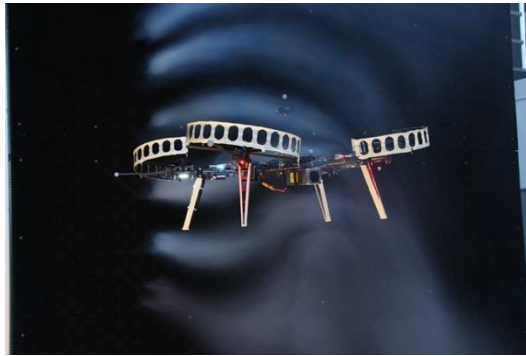


visualize a DNN [Li et al., NeurIPS 2018]

Model Learning: Challenges

❑ Challenge 3: Real-world models are NOT time-invariant

- Robustness: how do f_θ and the corresponding policy handle disturbances?
- Adaptability: how to fine-tune θ and the corresponding policy in real time?



❑ Challenge 4: High-dimensional state/action or rich observation settings

❑ Challenge 5: Under-fitting and compounding errors

- Hard to model complex/non-smooth dynamics (e.g., contacts), leads to worse asymptotic performance
- Compounding error happens in policy optimization

$$\min_u c_1(x_1, u_1) + c_2(f_\theta(x_1, u_1), u_2) + c_3(f_\theta(f_\theta(x_1, u_1), u_2), u_3) + \dots$$

Outline

(A big and very active field so we won't cover everything today)

- ❑ Regularized model learning + control (challenges 1 and 2)
 - ❑ Model learning with uncertainties: GP, ensemble (challenges 1 and 2)
 - ❑ MBRL adaptation: meta-MBRL (challenges 3)
 - ❑ Deep MBRL with rich observations: Dreamer and TD-MPC (challenges 4)
 - ❑ Methods to mitigate under-fitting and compounding errors (challenges 5)
-
- ❑ **Challenge 1:** Domain shift, over-fitting, and generalization
 - ❑ **Challenge 2:** “Good” model might be “bad” for control
 - ❑ **Challenge 3:** Real-world models are NOT time-invariant
 - ❑ **Challenge 4:** High-dimensional state/action or rich observation settings
 - ❑ **Challenge 5:** Under-fitting and compounding errors

Regularized Model Learning + Control

- General idea: not only minimize the regression error, also design a regularization term $J(\theta)$ (or a hard constraint $\theta \in \Theta$) such that
 - f_θ has some good properties for control (e.g., Lipschitz, smooth, controllable, etc)
 - f_θ matches the physics (e.g., Lagrangian systems, symmetric, invariance)

$$\min_{\theta} \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \|x_{t+1} - f_{\theta}(x_t, u_t)\|^2 + J(\theta)$$

$$\begin{aligned} & \min_{\theta} \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \|x_{t+1} - f_{\theta}(x_t, u_t)\|^2 \\ & \text{s. t. } \theta \in \Theta \end{aligned}$$

Regularized Model Learning + Control

- Example 1: residual dynamics learning for drone landing
 - Use spectral normalization (SN): $\|f_\theta(x, u)\|_{\text{Lip}} \leq \gamma$
 - γ is from control theory for stability guarantee

Theorem [Shi et al., ICRA'19]

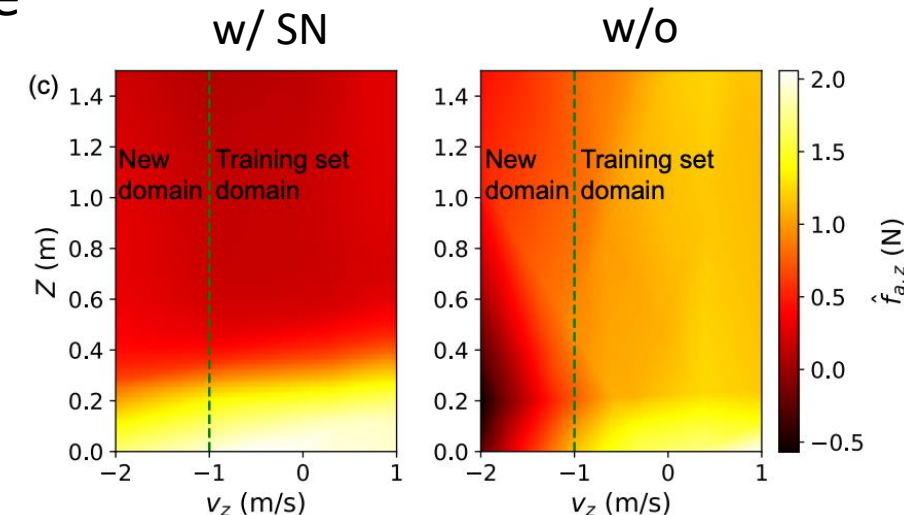
$$\|x - x_d\| \rightarrow C \cdot \frac{\epsilon}{\lambda_{\min}(K) - \gamma\rho} \text{ exponentially}$$

approximation error ($\|f - f_\theta\|_\infty$)
control gaintime delay

- DNN doesn't have good Lipschitz properties for free
 - w/ SN: $\|f_\theta\|_{\text{Lip}} \leq \gamma$
 - w/o SN: $\|f_\theta\|_{\text{Lip}} \approx 500\gamma$

**Neural Lander: Stable Drone Landing Control
Using Learned Dynamics**

Guanya Shi^{*,1}, Xichen Shi^{*,1}, Michael O'Connell^{*,1}, Rose Yu², Kamyar Azizzadenesheli³,
Animashree Anandkumar¹, Yisong Yue¹, and Soon-Jo Chung¹



Regularized Model Learning + Control

Example 2: learning stabilizable nonlinear dynamics

Learning Stabilizable Nonlinear Dynamics
with Contraction-Based Regularization

Sumeet Singh¹, Spencer M. Richards¹, Vikas Sindhwani², Jean-Jacques E. Slotine³, and
Marco Pavone¹

$$\min_{\hat{f} \in \mathcal{H}} \sum_{i=1}^N \left\| \hat{f}(x_i, u_i) - \dot{x}_i \right\|_2^2 + \mu \|\hat{f}\|_{\mathcal{H}}^2$$

s.t. $\hat{f}$ is stabilizable,

Example 3: deep Lagrangian networks

DEEP LAGRANGIAN NETWORKS:
USING PHYSICS AS MODEL PRIOR FOR DEEP LEARNING

Michael Lutter, Christian Ritter & Jan Peters *

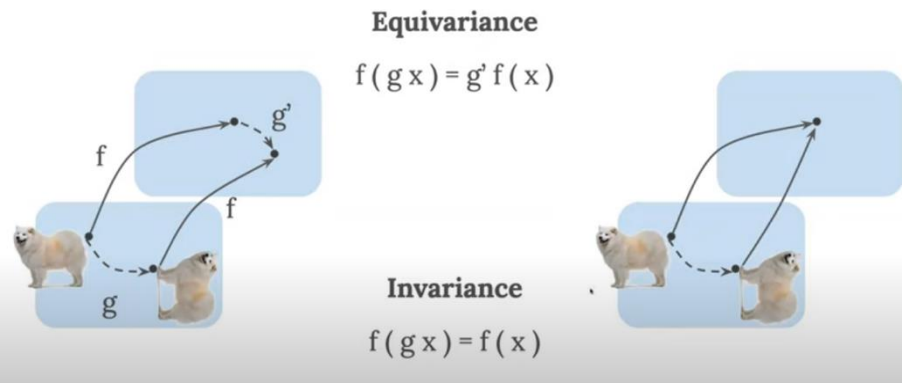
$$(\theta^*, \psi^*) = \arg \min_{\theta, \psi} \ell \left(\hat{f}^{-1}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}; \theta, \psi), \boldsymbol{\tau} \right)$$

with $\hat{f}^{-1}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}; \theta, \psi) = \hat{\mathbf{L}} \hat{\mathbf{L}}^T \ddot{\mathbf{q}} + \frac{d}{dt} (\hat{\mathbf{L}} \hat{\mathbf{L}}^T) \dot{\mathbf{q}} - \frac{1}{2} \left(\frac{\partial}{\partial \mathbf{q}} (\dot{\mathbf{q}}^T \hat{\mathbf{L}} \hat{\mathbf{L}}^T \dot{\mathbf{q}}) \right)^T + \hat{\mathbf{g}}$

s.t. $0 < \mathbf{x}^T \hat{\mathbf{L}} \hat{\mathbf{L}}^T \mathbf{x} \quad \forall \mathbf{x} \in \mathbb{R}_0^n$

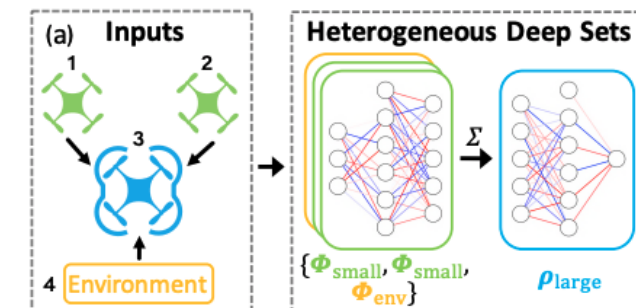
Example 4: Invariance and equivariance

- Dive deeper: geometric deep learning and geometric control



Neural-Swarm2: Planning and Control of Heterogeneous Multirotor Swarms using Learned Interactions

Guanya Shi, Wolfgang Hönig, Xichen Shi, Yisong Yue, and Soon-Jo Chung



Model Learning with Uncertainties

- ❑ General idea: f_θ is a *probabilistic* model, which can predict uncertainties
- ❑ Why better?
 - Discourage overconfidence by jointly learning the uncertainties
 - Use this uncertainties for uncertainty-aware control
- ❑ Popular scheme 1: Gaussian Processes (GP)
 - A non-parametric learning method
 - μ, Σ for a test data point are expressed by all training data
 - Universal approximator for functions in RKHS

$$\mu_a^d(z) = K_{zz}^a (K_{zz}^a + I\sigma_a^2)^{-1} [\mathbf{y}]_a$$

$$\Sigma_a^d(z) = K_{zz}^a - K_{zz}^a (K_{zz}^a + I\sigma_a^2)^{-1} K_{zz}^a$$

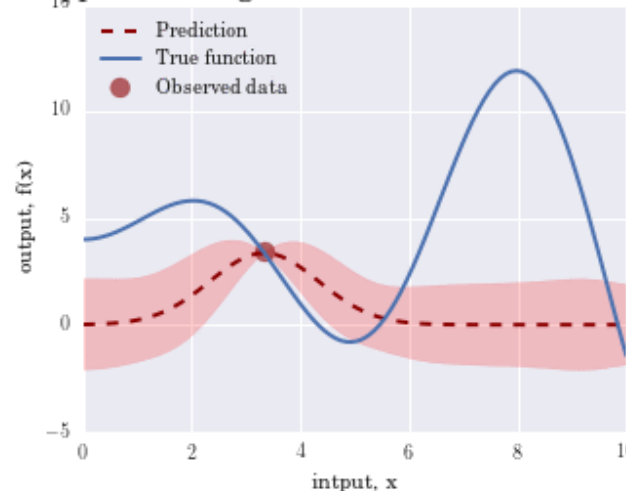
$$k^a(z_i, z_j) = \sigma_{f,a}^2 \exp\left(-\frac{1}{2}(z_i - z_j)^\top L_a^{-1}(z_i - z_j)\right)$$

$\mathbf{z}$: training data

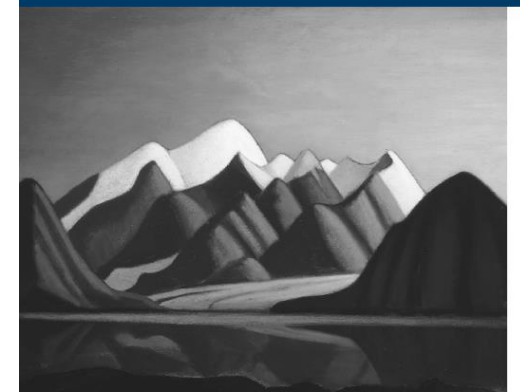
z : test data point

k : kernel function

Approximating true function with more data



Gaussian Processes for
Machine Learning



Carl Edward Rasmussen and Christopher K. I. Williams

Model Learning with Uncertainties

□ Example 1: Use both μ and Σ for policy search

PILCO: A Model-Based and Data-Efficient Approach to Policy Search

Marc Peter Deisenroth

Department of Computer Science & Engineering, University of Washington, USA

MARC@CS.WASHINGTON.EDU

Carl Edward Rasmussen

Department of Engineering, University of Cambridge, UK

CER54@CAM.AC.UK

Cart pole performance using 17.5s data



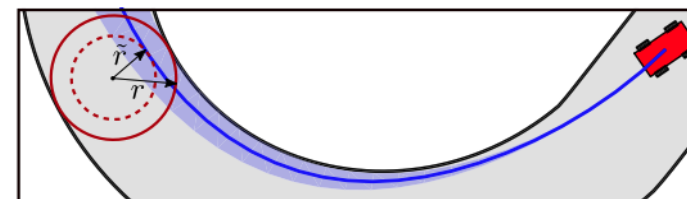
□ Example 2: Use both μ and Σ for MPC

- Need uncertainty propagation

$$\begin{aligned} \min_{\Pi} \quad & \mathbb{E} \left(l_f(x_N) + \sum_{i=0}^{N-1} l_i(x_i, u_i) \right) \\ \text{s.t.} \quad & x_{i+1} = f(x_i, u_i) + B_d(d(x_i, u_i) + w_i) \\ & u_i = \pi_i(x_i) \\ & \Pr(x_{i+1} \in \mathcal{X}_{i+1}) \geq p_x \\ & \Pr(u_i \in \mathcal{U}_i) \geq p_u \\ & x_0 = x(k) \end{aligned}$$

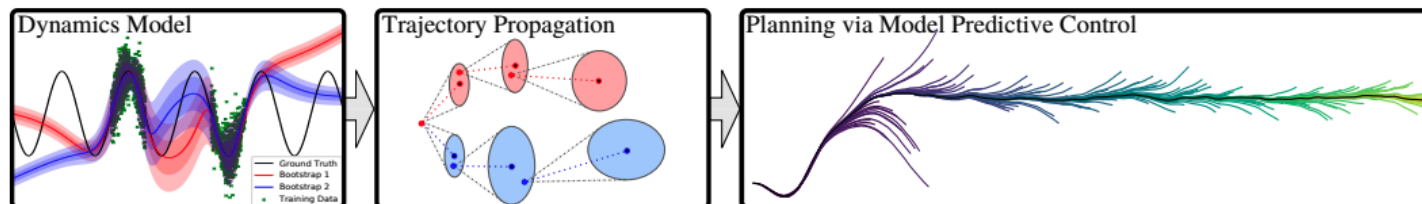
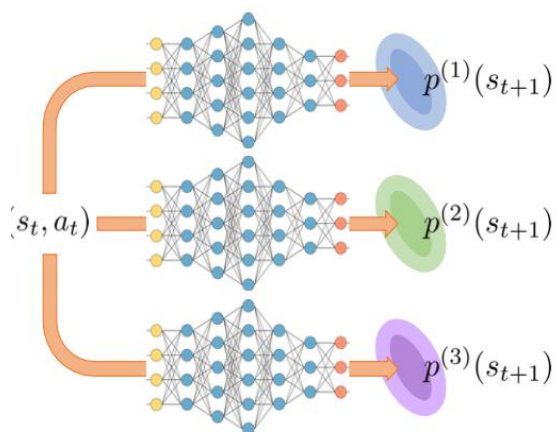
$$\begin{aligned} \mu_{i+1}^x &= f(\mu_i^x, \mu_i^u) + B_d \mu^d(\mu_i^x, \mu_i^u) \\ \Sigma_{i+1}^x &= \tilde{A}_i \Sigma_i^x \tilde{A}_i^T + B_d(\Sigma^d(\mu_i^x, \mu_i^u) + \Sigma^w) B_d^T \end{aligned}$$

Cautious Model Predictive Control Using Gaussian Process Regression



Model Learning with Uncertainties

- ❑ Popular scheme 2: Ensemble-based methods
 - Learn multiple model: $f_{\theta_1}, \dots, f_{\theta_N}$
 - Use their “disagreements” for uncertainty
- ❑ Example: PETS (Probabilistic Ensembles with Trajectory Sampling)
 - Use CEM for planning



Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models

Kurtland Chua Roberto Calandra Rowan McAllister Sergey Levine
 Berkeley Artificial Intelligence Research
 University of California, Berkeley
 {kchua, roberto.calandra, rmcallister, svlevine}@berkeley.edu

Algorithm 1 Our model-based MPC algorithm ‘PETS’:

- 1: Initialize data $\mathbb{D}$ with a random controller for one trial.
- 2: **for** Trial $k = 1$ to K **do**
- 3: Train a *PE* dynamics model $\tilde{f}$ given $\mathbb{D}$.
- 4: **for** Time $t = 0$ to TaskHorizon **do**
- 5: **for** Actions sampled $\mathbf{a}_{t:t+T} \sim \text{CEM}(\cdot)$, 1 to NSamples **do**
- 6: Propagate state particles $\mathbf{s}_\tau^p$ using *TS* and $\tilde{f}|\{\mathbb{D}, \mathbf{a}_{t:t+T}\}$.
- 7: Evaluate actions as $\sum_{\tau=t}^{t+T} \frac{1}{P} \sum_{p=1}^P r(\mathbf{s}_\tau^p, \mathbf{a}_\tau)$
- 8: Update $\text{CEM}(\cdot)$ distribution.
- 9: Execute first action $\mathbf{a}_t^*$ (only) from optimal actions $\mathbf{a}_{t:t+T}^*$.
- 10: Record outcome: $\mathbb{D} \leftarrow \mathbb{D} \cup \{s_t, \mathbf{a}_t^*, s_{t+1}\}$.

Multi-task and Adaptive MBRL

- ❑ General idea: adapt model parameters in real time
 - Policy will be adapted accordingly
- ❑ Connections with adaptive control:
 - Can be viewed as adaptive optimal control

	Continuous	Discrete
System	$\dot{x} = f(x, u, a(t))$	$x_{t+1} = f(x_t, u_t, a_t)$
Policy	$u = \pi(x, \hat{a}; \theta_\pi)$	$u_t = \pi(x_t, \hat{a}_t; \theta_\pi)$
Adaptation	$\dot{\hat{a}} = \mathcal{A}(\hat{a}(0:t), x(0:t), u(0:t); \theta_{\mathcal{A}})$	$\hat{a}_{t+1} = \mathcal{A}(\hat{a}_{1:t}, x_{1:t}; u_{1:t}; \theta_{\mathcal{A}})$

- ❑ Learning tools: online learning, meta-learning, representation learning
- ❑ **Key** (learning-to-learn): in the training phase, learn θ such that it is “adaptable”

$$\begin{aligned} \min_{\theta} & \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \|x_{t+1} - f_{\theta}(x_t, u_t)\|^2 \\ \text{s. t.} & \quad \theta + \delta\theta \text{ is good in new environments} \end{aligned}$$

Multi-task and Adaptive MBRL

- Example 1: meta-learning dynamics + real-time dynamics adaptation + MPPI

LEARNING TO ADAPT IN DYNAMIC, REAL-WORLD ENVIRONMENTS THROUGH META-REINFORCEMENT LEARNING

Anusha Nagabandi*, Ignasi Clavera*, Simin Liu,
Ronald S. Fearing, Pieter Abbeel, Sergey Levine, & Chelsea Finn
University of California, Berkeley
{nagaban2, iclavera, simin.liu}@berkeley.edu
{ronf, pabbeel, svlevine, cbfinn}@berkeley.edu

Algorithm 2 Online Model Adaptation
(test time)

Require: Meta-learned parameters θ_*, ψ_*

Require: controller(), H, r, n_A

1: $\mathcal{D} \leftarrow \emptyset$

2: **for** each timestep t **do**

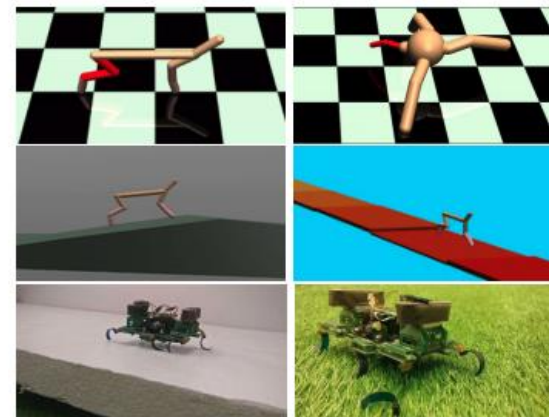
3: $\theta'_* \leftarrow u_{\psi_*}(\mathcal{D}(t-M, t-1), \theta_*)$

4: $\mathbf{a} \leftarrow \text{controller}(\theta'_*, r, H, n_A)$

5: Execute $\mathbf{a}$, add result to $\mathcal{D}$

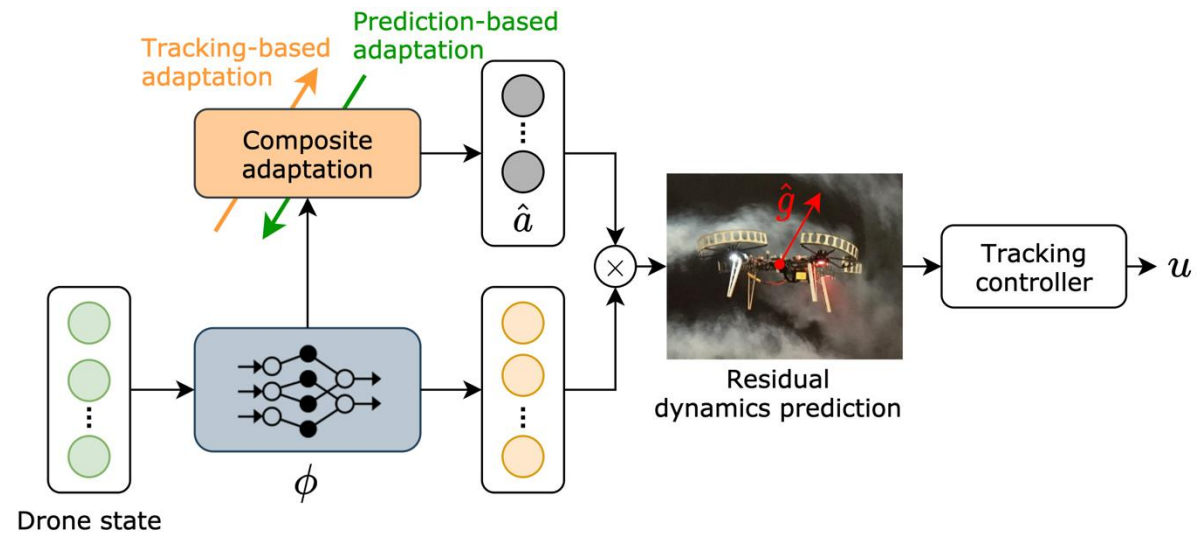
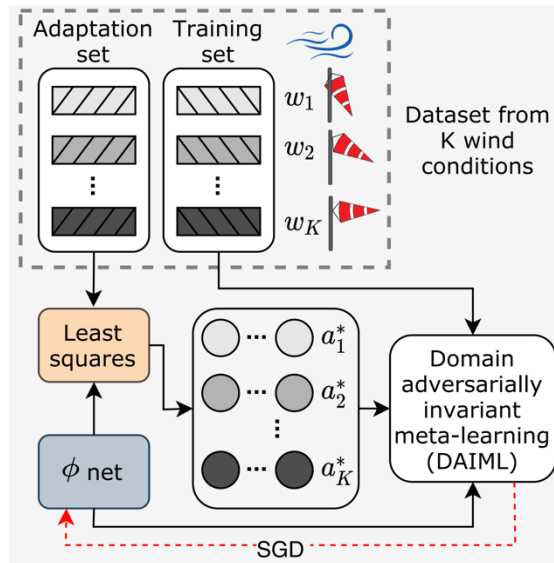
6: **end for**

7: Return rollout $\mathcal{D}$



Multi-task and Adaptive MBRL

Example 2: meta-learning dynamics + adaptive control



Neural-Fly Enables Rapid Learning for Agile Flight in Strong Winds

Michael O'Connell[†], Guanya Shi[†], Xichen Shi,
Kamyar Azizzadenesheli, Anima Anandkumar, Yisong Yue, Soon-Jo Chung*

Division of Engineering and Applied Science, California Institute of Technology

[†] The first two authors contributed equally to this article. Alphabetical order.

* Corresponding authors. Email: sjchung@caltech.edu

This is the accepted version of Science Robotics Vol. 7, Issue 66, eabm6597 (2022)

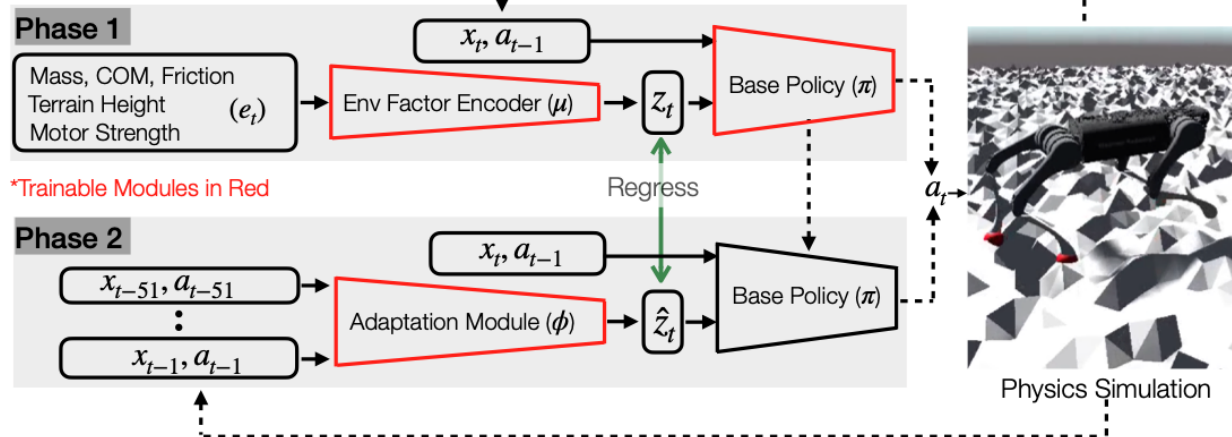
DOI: 10.1126/scirobotics.abm6597 Video: <https://youtu.be/TuF9teCZX0U>

Data and training code: <https://github.com/aerorobotics/neural-fly>

Multi-task and Adaptive MBRL

- Example 3: policies $u = \pi(x_t, \theta_t)$ conditioned on model parameters
 - θ_t can be viewed as a model abstraction
 - θ_t is either online estimated or predicted by DNNs
 - Somewhere between MBRL and MFRL

A) Training in Simulation



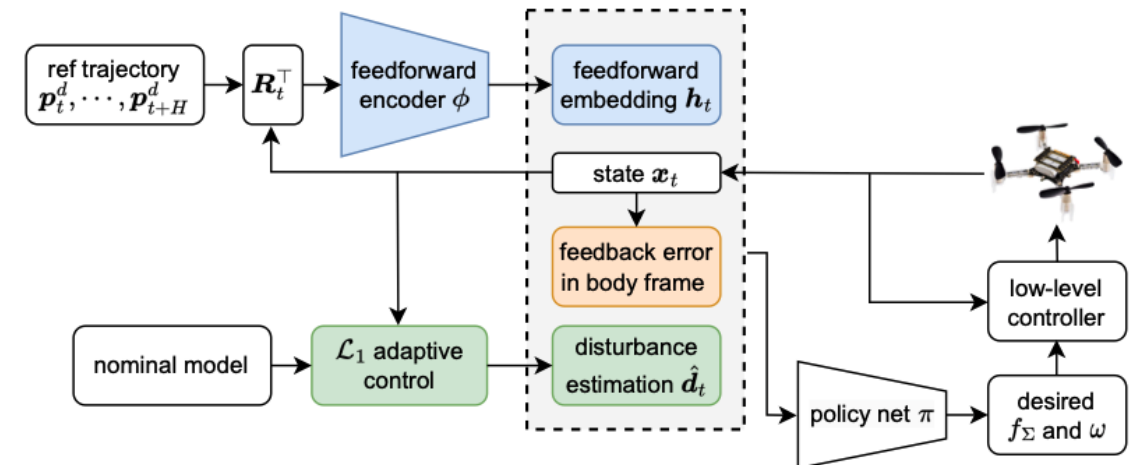
RMA: Rapid Motor Adaptation for Legged Robots

Ashish Kumar
UC Berkeley

Zipeng Fu
Carnegie Mellon University

Deepak Pathak
Carnegie Mellon University

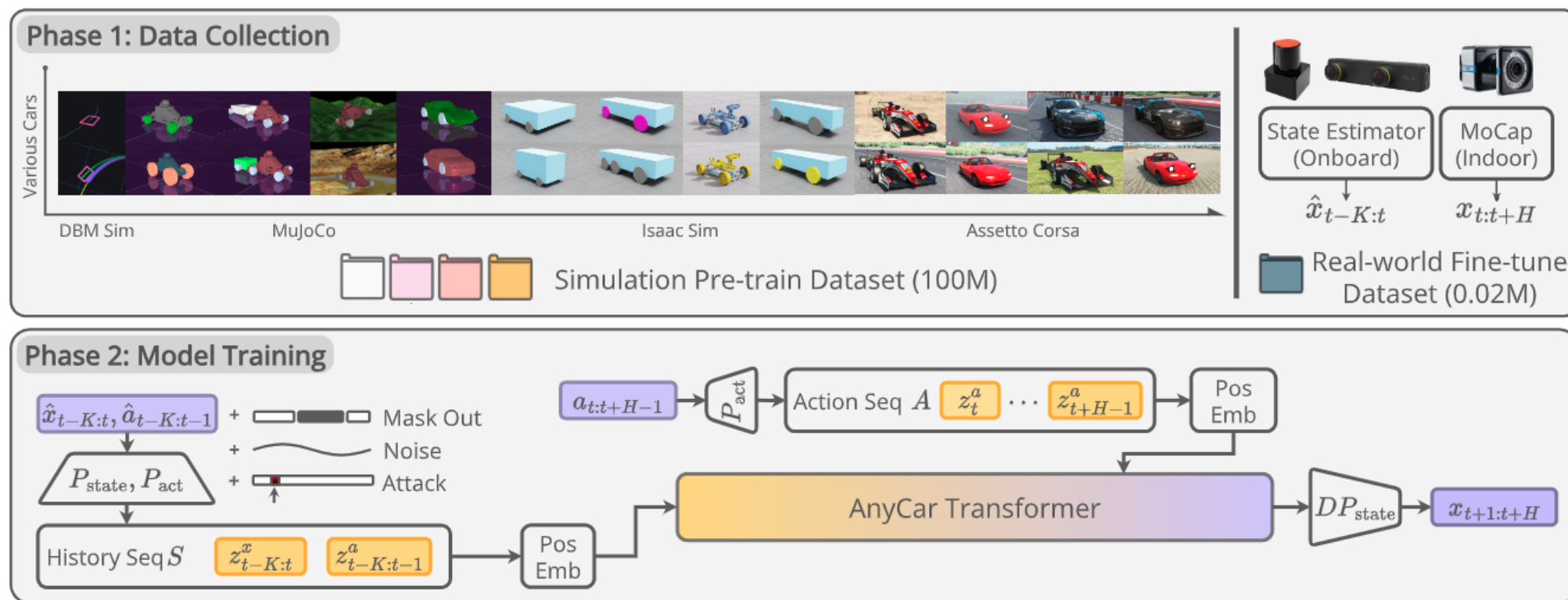
Jitendra Malik
UC Berkeley, Facebook



DATT: Deep Adaptive Trajectory Tracking, CoRL (oral), 2023

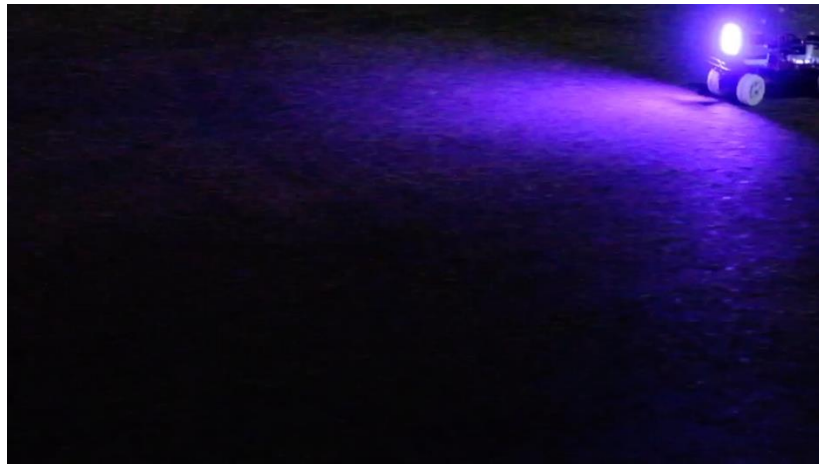
Multi-task and Adaptive MBRL

- ❑ Example 4: history-conditioned model and in-context adaptation
 - Learn $f_{\theta}(x_t, u_t, \text{history of } x \text{ and } u)$
- ❑ AnyCar: large-scale sim data (100M) and few-shot real data for fine-tuning (0.02M)
- ❑ Robust training: mask out, add noise, attack



Multi-task and Adaptive MBRL

- ❑ AnyCar: large-scale sim data (100M) and few-shot real data for fine-tuning (0.02M)
- ❑ Robust training: mask out, add noise, attack
- ❑ Sampling-based MPC for control



Drifting in the wild
(w/ plastic tires)

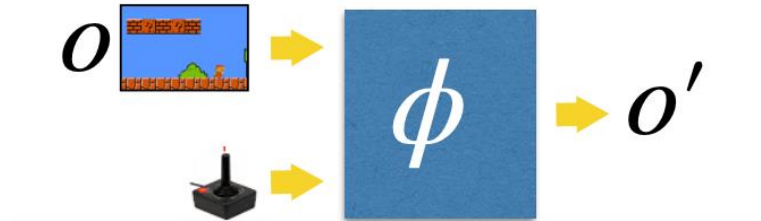


Precise tracking with rapid adaptation



Deep MBRL for Rich Observations

- Goal: Learning world model from high-dimensional sensory input (e.g., images)



- Intersection between computer vision and MBRL
- Example: Dreamer

DREAM TO CONTROL: LEARNING BEHAVIORS BY LATENT IMAGINATION

Danijar Hafner *
University of Toronto
Google Brain

Timothy Lillicrap
DeepMind

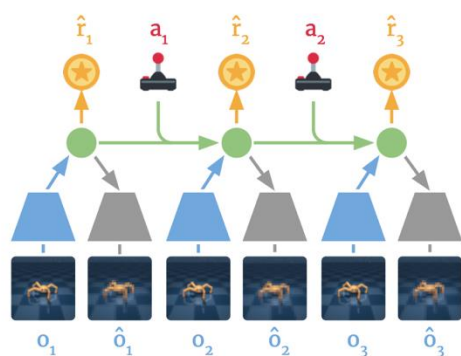
Jimmy Ba
University of Toronto

Mohammad Norouzi
Google Brain

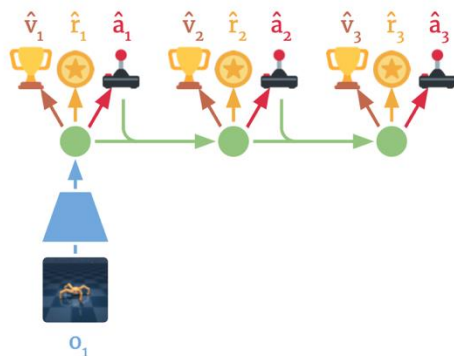
Deep MBRL for Rich Observations

❑ Dreamer-v1

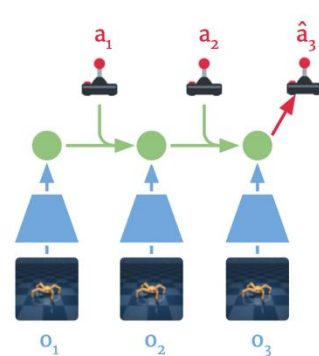
- Learning dynamics model in a compact latent space
- Learning with 3 objective: *reward prediction*, *state reconstruction* and *contrastive estimation* (mutual information)
- Actor-critic method on imaginary trajectories for behavior learning
- Repeat dynamics learning and behavior learning until convergence



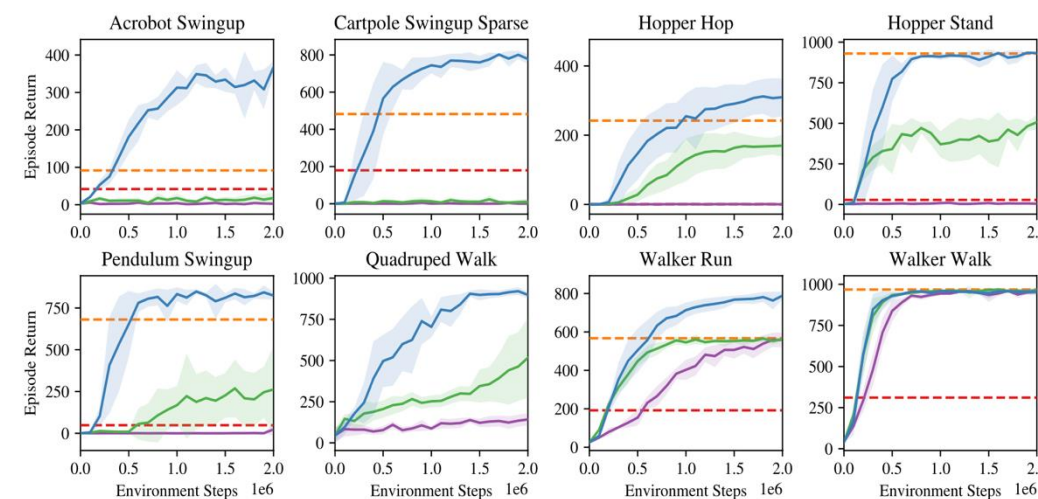
(a) Learn dynamics from experience



(b) Learn behavior in imagination



(c) Act in the environment



Representation model:

Transition model:

Reward model:

$$p(s_t | s_{t-1}, a_{t-1}, o_t)$$

$$q(s_t | s_{t-1}, a_{t-1})$$

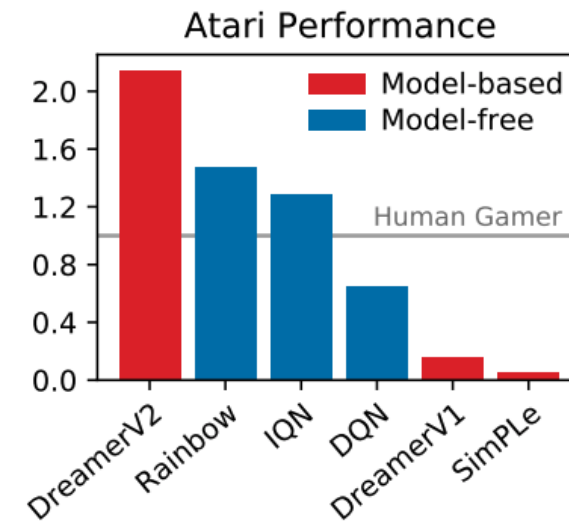
$$q(r_t | s_t).$$

— Dreamer — No value — PlaNet — D4PG (1e9 steps) - - A3C (1e9 steps, proprio)

Deep MBRL for Rich Observations

❑ Dreamer-v2

- Improved version for multi-task learning
- Idea: Learn a world model with discrete representations for different tasks



❑ Dreamer-v3

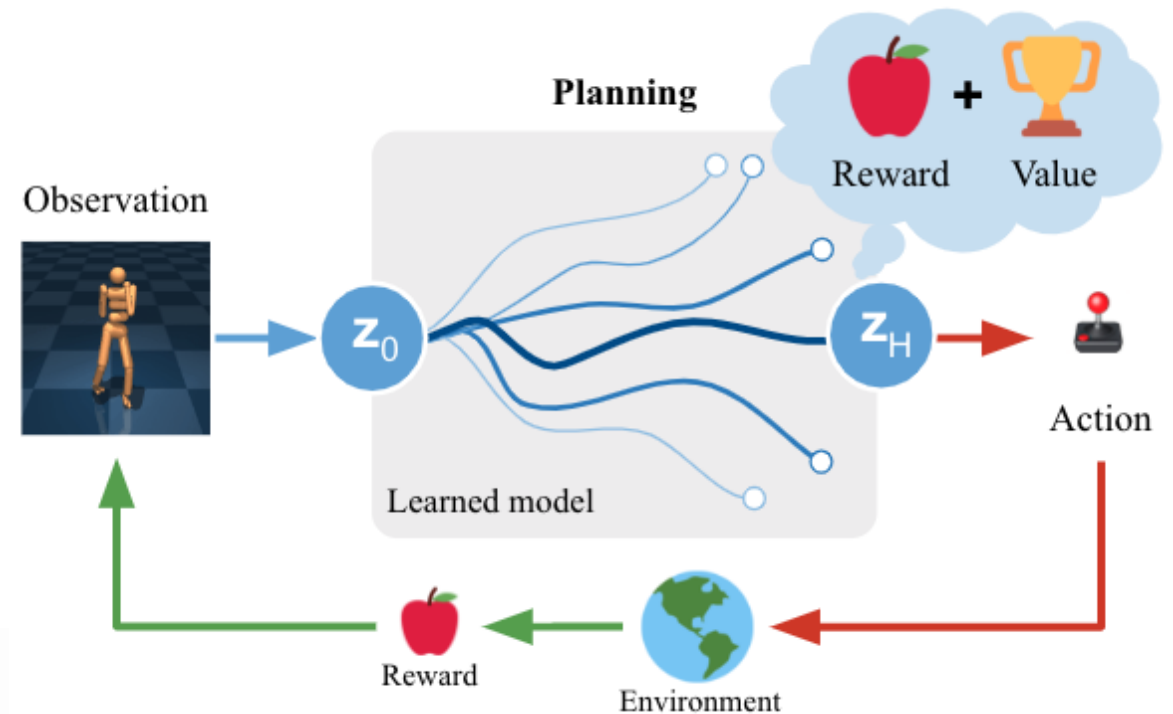
- Improved version for better scalability
- Diverse domains (not just Atari)

Deep MBRL for Rich Observations

□ TD-MPC

- Learn a latent dynamics and reward model
- No reconstruction loss – reward is the only learning signal
- MPC (using MPPI) + learned cost-to-go (value) functions for inference

$$\mathbb{E}_{\Gamma} \left[\underbrace{\gamma^H Q_{\theta}(\mathbf{z}_H, \mathbf{a}_H)}_{\text{Value}} + \underbrace{\sum_{t=0}^{H-1} \gamma^t R_{\theta}(\mathbf{z}_t, \mathbf{a}_t)}_{\text{Rewards}} \right]$$



Temporal Difference Learning for Model Predictive Control

Deep MBRL for Rich Observations

- TD-MPC2
 - Multi-task and scalable version of TD-MPC

TD-MPC2:

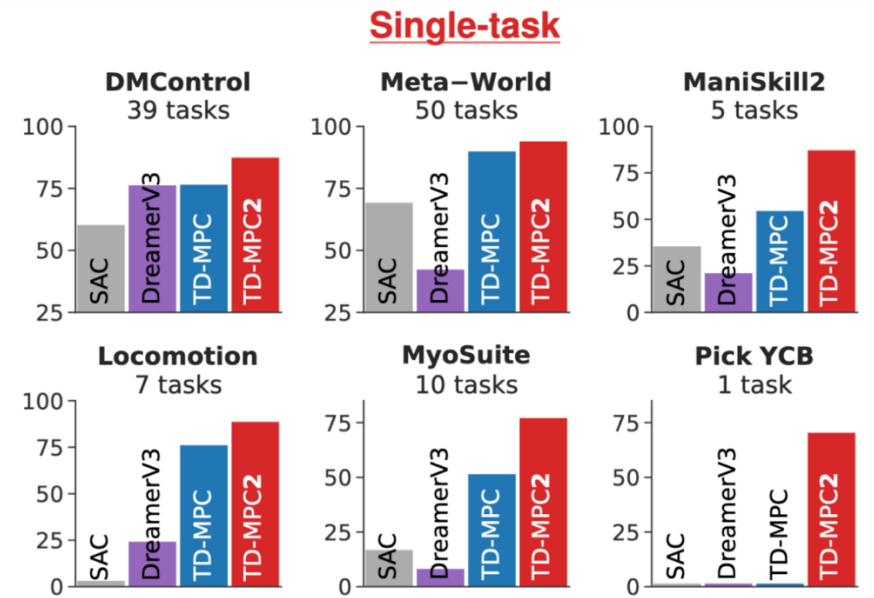
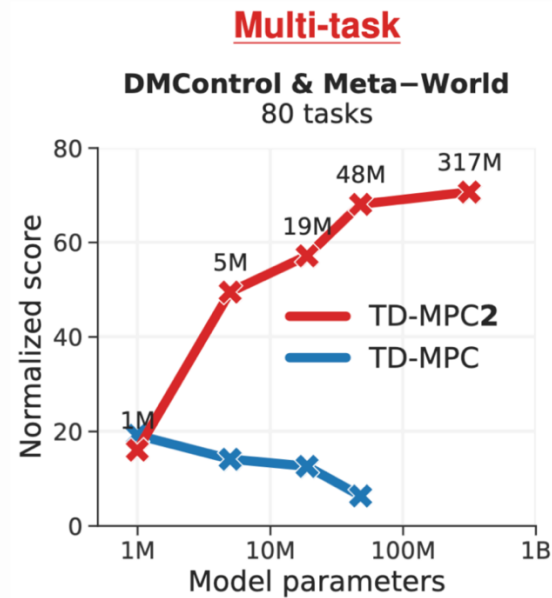
Scalable, Robust World Models for Continuous Control

ICLR 2024 **Spotlight**

Nicklas Hansen, Hao Su*, Xiaolong Wang*

UC San Diego

*Equal advising



Deep MBRL for Rich Observations

□ Diffusion Model Predictive Control (D-MPC)

- Learn multi-step “action proposal” and multi-step dynamics model, both using diffusion models

3.2. Model learning

We assume access to an offline dataset of trajectories, consisting of (state, action, reward) triples: $\mathcal{D} = \{s_{1:T_1}^1, a_{1:T_1}^1, r_{1:T_1}^1, s_{1:T_2}^2, a_{1:T_2}^2, r_{1:T_2}^2, \dots, s_{1:T_M}^M, a_{1:T_M}^M, r_{1:T_M}^M\}$. We then use this to fit a diffusion-based dynamics model $p_d(s_{t+1:t+F}|s_t, h_t, a_{t:t+F-1})$ and another diffusion-based action proposal $\rho(a_{t:t+F-1}|s_t, h_t)$. To fit these models, we minimize the denoising score matching loss in the usual way. We include a detailed review of diffusion model training in Appendix A, and refer the readers to e.g. [\[Kar+22\]](#) for additional discussions.

Diffusion Model Predictive Control

Guangyao Zhou¹, Sivaramakrishnan Swaminathan¹, Rajkumar Vasudeva Raju¹, J. Swaroop Guntupalli¹, Wolfgang Lehrach¹, Joseph Ortiz¹, Antoine Dedieu¹, Miguel Lázaro-Gredilla¹ and Kevin Murphy¹

Underfitting and Compounding Errors

- ❑ Underfitting: Hard to model complex/non-smooth dynamics (e.g., contacts), leads to worse asymptotic performance
- ❑ Compounding error: $\min_u c_1(x_1, u_1) + c_2(f_\theta(x_1, u_1), u_2) + c_3(f_\theta(f_\theta(x_1, u_1), u_2), u_3) + \dots$
- ❑ Solution 1: Hierarchical or hybrid model
 - E.g., $f_{\theta_1}(x, u)$ w/o contact and $f_{\theta_2}(x, u)$ with contact
- ❑ Solution 2: Better representation
 - E.g., learning dynamics in the latent space (Dreamer-style)
- ❑ Solution 3: Curriculum learning
 - E.g., Curriculum on the prediction horizon

$$\mathcal{L}_K(\theta) = \frac{1}{2K} \sum_i \sum_t \sum_{k=1}^K \left\| \hat{\mathbf{x}}_{t+k}^{(i)} - \mathbf{x}_{t+k}^{(i)} \right\|^2$$

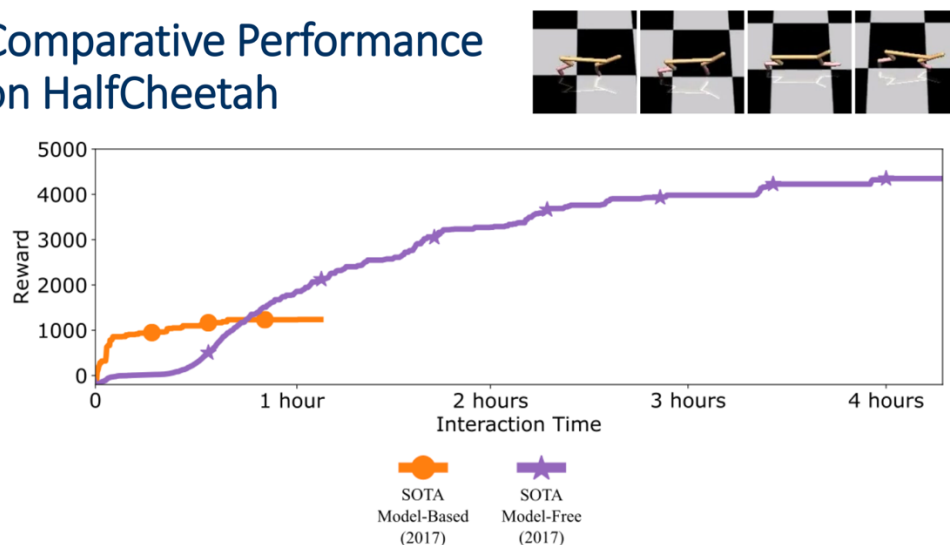
Action-Conditional Video Prediction using Deep Networks in Atari Games

Junhyuk Oh Xiaoxiao Guo Honglak Lee Richard Lewis Satinder Singh
University of Michigan, Ann Arbor, MI 48109, USA
{junhyuk, guoxiao, honglak, rickl, baveja}@umich.edu

Underfitting and Compounding Errors

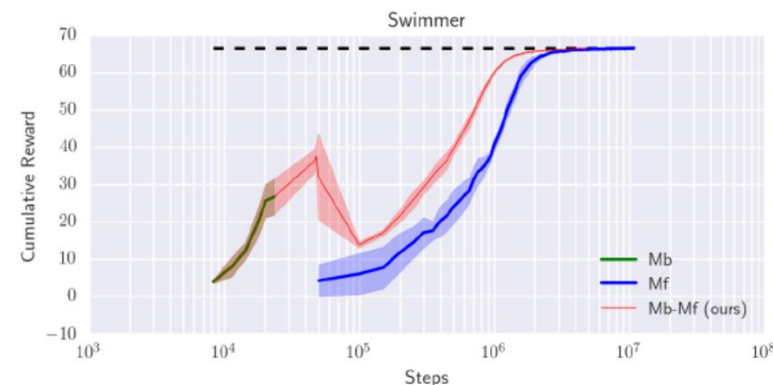
- ❑ Underfitting: Hard to model complex/non-smooth dynamics (e.g., contacts), leads to worse asymptotic performance
- ❑ Compounding error: $\min_u c_1(x_1, u_1) + c_2(f_\theta(x_1, u_1), u_2) + c_3(f_\theta(f_\theta(x_1, u_1), u_2), u_3) + \dots$
- ❑ Solution 4: Fine-tuned by MFRL methods

Comparative Performance
on HalfCheetah



Neural Network Dynamics
for Model-Based Deep Reinforcement Learning
with Model-Free Fine-Tuning

Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, Sergey Levine
University of California, Berkeley



Summary: How to Learn f_θ ?

$$\min_{\theta} \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \|x_{t+1} - f_\theta(x_t, u_t)\|^2$$

- ❑ Regularized model learning + control (challenges 1 and 2)
 - ❑ Model learning with uncertainties: GP, ensemble (challenges 1 and 2)
 - ❑ MBRL adaptation: meta-MBRL (challenges 3)
 - ❑ Deep MBRL with rich observations: Dreamer (challenges 4)
 - ❑ Methods to mitigate under-fitting and compounding errors (challenges 5)
-
- ❑ **Challenge 1:** Domain shift, over-fitting, and generalization
 - ❑ **Challenge 2:** “Good” model might be “bad” for control
 - ❑ **Challenge 3:** Real-world models are NOT time-invariant
 - ❑ **Challenge 4:** High-dimensional state/action or rich observation settings
 - ❑ **Challenge 5:** Under-fitting and compounding errors

Summary: How to Use f_θ for Control?

$$\min_{\theta} \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \|x_{t+1} - f_\theta(x_t, u_t)\|^2$$

- ❑ Method 1: Policy search
- ❑ Method 2: MPC
- ❑ Method 3: Use f_θ as a simulator for MFRL
 - Dyna-style algorithms

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \epsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

**Dyna, an Integrated Architecture
for Learning, Planning, and Reacting**

Richard S. Sutton
GTE Laboratories Incorporated
Waltham, MA 02254
sutton@gte.com

World Model

❑ Philosophical question: Is world model the next frontier in our path to AGI?

LeCun's slide

Modular Architecture for Autonomous AI

Y. LeCun

► Configurator

- Configures other modules for task

► Perception

- Estimates state of the world

► World Model

- Predicts future world states

► Cost

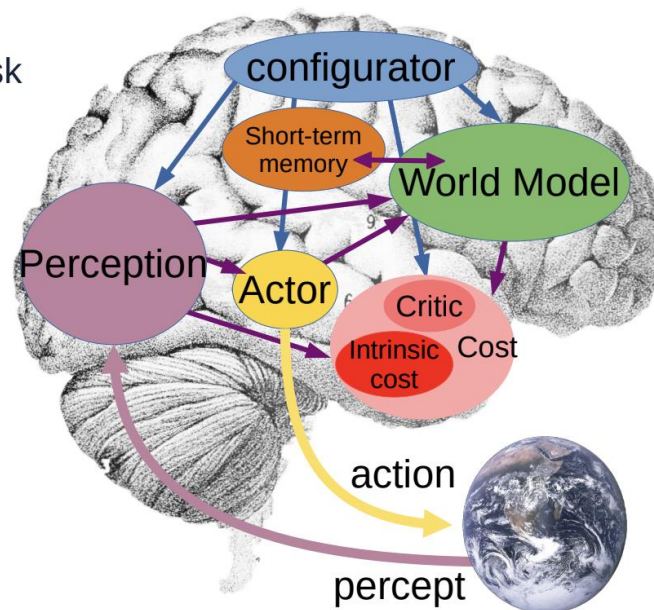
- Compute "discomfort"

► Actor

- Find optimal action sequences

► Short-Term Memory

- Stores state-cost episodes



Yann LeCun
@ylecun

...

Indeed, I do favor MPC over RL.

I've been making that point since at least 2016.

RL requires ridiculously large numbers of trials to learn any new task.

In contrast MPC is zero shot: If you have a good world model and a good task objective, MPC can solve new tasks without any task-specific learning.

That's the magic of planning.

It doesn't mean that RL is useless, but its use should be a last resort.

World Model

- ❑ OpenAI Sora can be viewed as a “preliminary” large world model (not conditional on actions)

Creating video from text

Sora is an AI model that can create realistic and imaginative scenes from text instructions.

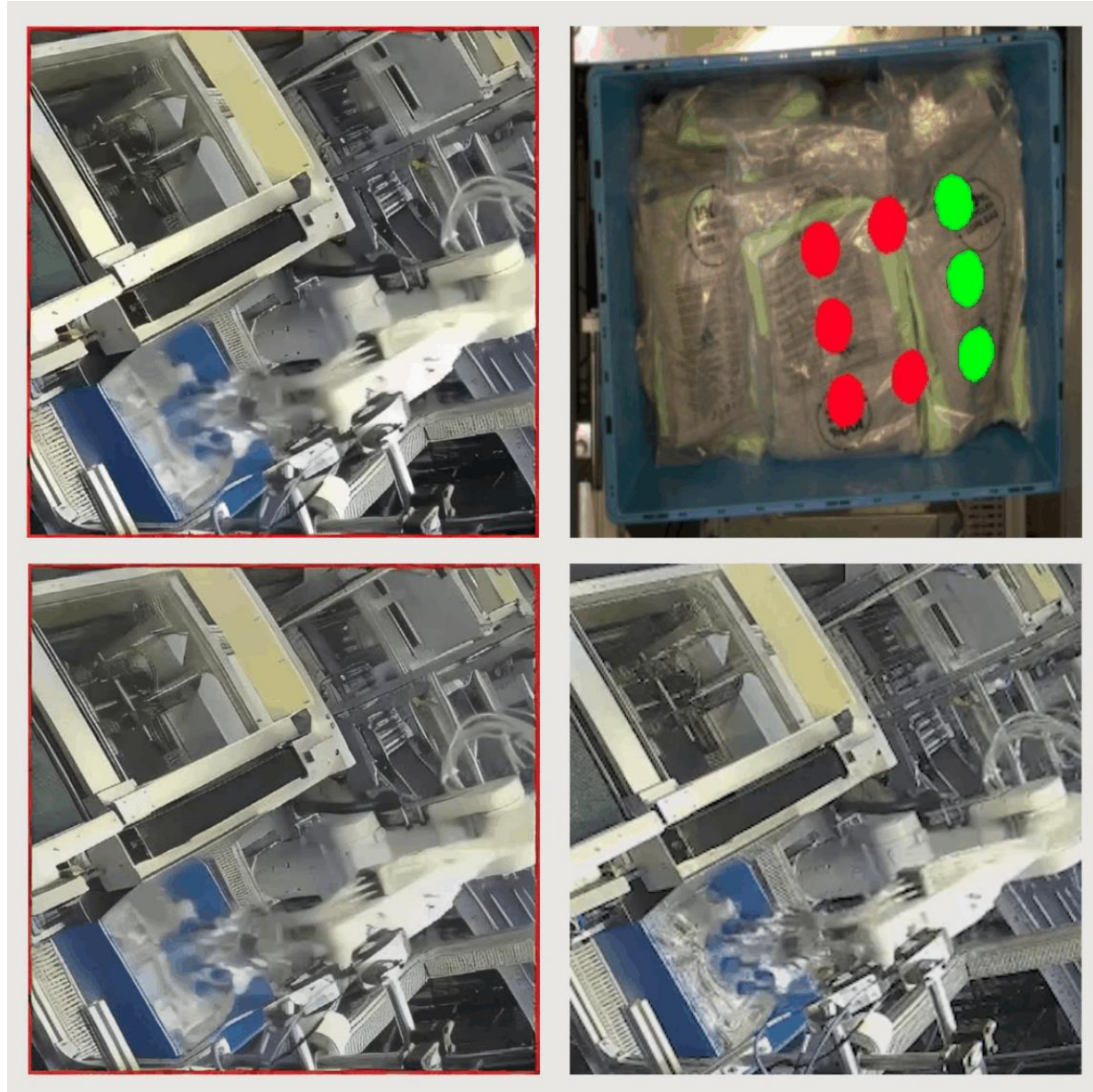


Robotics Foundation Model

- ❑ RFM-1: Robotics Foundation Model
- ❑ World models conditional on actions!

RFM-1 simulates a picking action (bottom left) based on an initial image (top left) and prescribed item to pick (top right). The actual real-world pick result is on the bottom right.

covariant

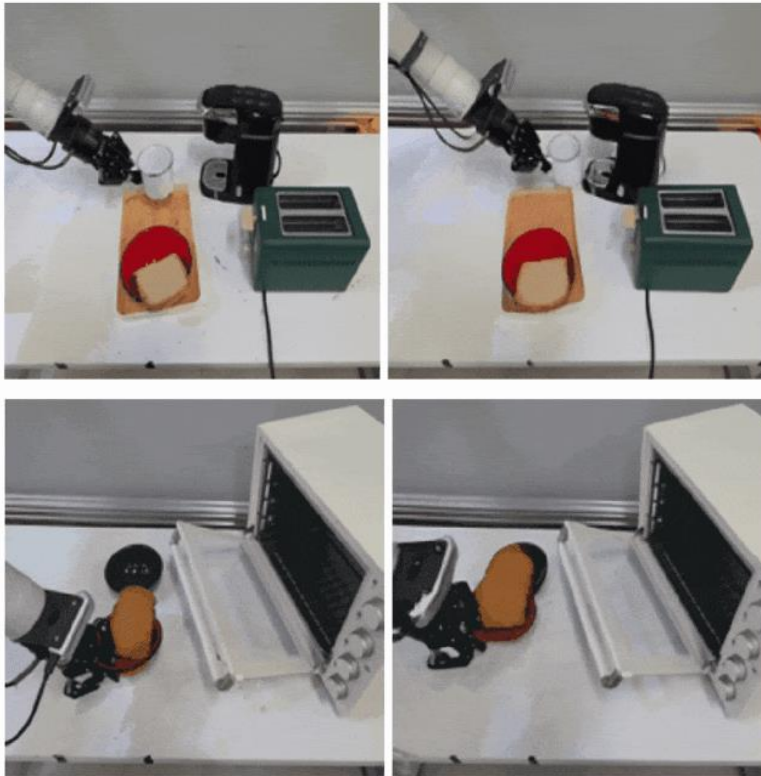


Robotics Foundation Model

- GR-2: V-L pretraining (from internet videos) and V-L-A fine-tuning (on robot data)

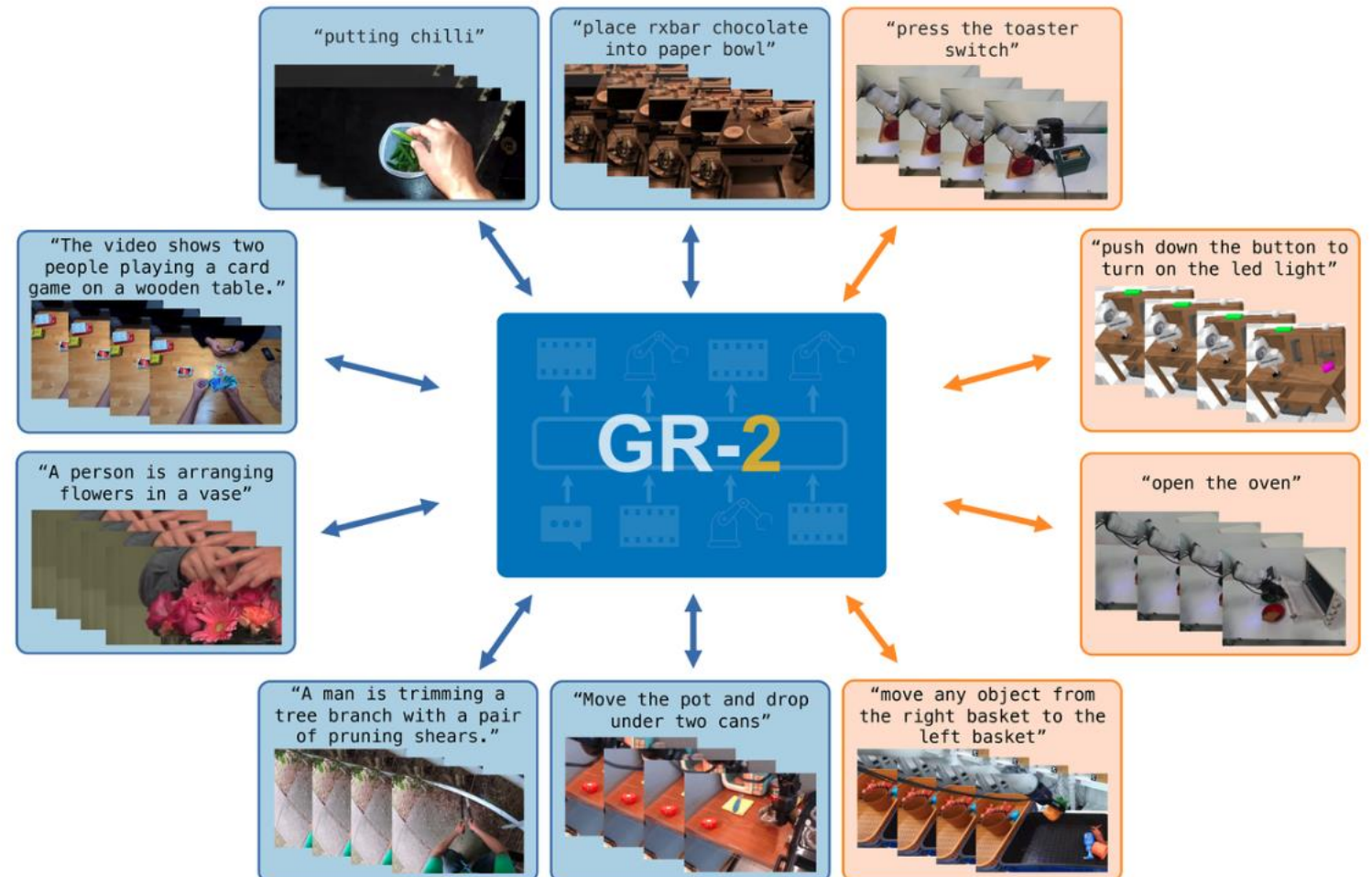
Ground Truth

Prediction



Video-Language Pre-training

Video-Language-Action Fine-tuning



Summary of Model-based RL and Control

- ❑ Part 1: control theory basics
 - Stability and Lyapunov methods for stability
 - Uncertainty and robust control
 - Adaptive control
 - Linear systems and continuous-time LQR
 - Linearization of nonlinear systems
- ❑ Part 2: how can we make decisions if we *know* the dynamics?
 - Trajectory optimization (TO) and model predictive control (MPC)
 - Discrete time LQR
 - Sequential quadratic/convex programming (SQP/SCP)
 - iLQR/DDP
 - Sampling-based MPC
- ❑ Part 3: learn *unknown* dynamics and use it for control
 - Model learning
 - Policy optimization with the learned model
 - Deep MBRL: Dreamer, TD-MPC, MBRL+MFRL, etc

Thank You!